

REDIS_CPP BENCHMARK & ANALYSIS REPORT

A High-Performance C++20 Key-Value Store implementing the Reactor Pattern

Date: 2026-07-12 | OS: macOS (kqueue) | Test Tool: redis-benchmark

1. Executive Summary

redis_cpp is a lightweight, high-performance, Redis-compatible in-memory key-value store written in modern C++20. The project was redesigned and rewritten from a legacy C codebase (ev_kv_store) to leverage modern object-oriented programming paradigms, type-safe structures, and efficient operating system event-loop APIs.

By replacing blocking thread-per-connection patterns with an event-driven Reactor pattern (leveraging epoll on Linux and kqueue on macOS), redis_cpp achieves massive scalability with low CPU and memory footprint. Business logic and command execution are completely decoupled from the I/O reactor thread via an asynchronous FIFO Thread Pool. Dynamic memory allocation is managed using robust RAIL socket wrappers and smart pointers, eliminating memory leaks.

Key Technical Highlights:

- Throughput: Natively achieves over 87k RPS for GET operations using pipelining.
- Efficiency: Min-heap $O(\log N)$ TTL expiry scan, replacing linear $O(N)$ scans.
- Reliability: Append-Only File (AOF) persistence with full system recovery on startup.

2. Benchmark Results

Command / Test	Mode	Throughput (RPS)	Avg Latency (ms)	p50 Latency (ms)	p99 Latency (ms)
PING_INLINE	Single Request	29,994	0.978 ms	0.895 ms	2.423 ms
PING_MBULK	Single Request	23,207	1.451 ms	0.943 ms	9.391 ms
SET	Single Request	24,509	1.341 ms	0.999 ms	5.751 ms
GET	Single Request	25,432	1.289 ms	0.951 ms	8.183 ms
SET (with EX 10s)	Single Request (Custom)	29,542	0.988 ms	0.927 ms	1.895 ms
SET (Pipelined)	Pipeline=16	81,300	3.306 ms	2.623 ms	12.831 ms
GET (Pipelined)	Pipeline=16	87,489	3.085 ms	2.727 ms	7.583 ms

3. Performance & Concurrency Analysis

3.1 Impact of Pipelining

During standard, synchronous single-request testing, throughput is capped at around 30k RPS due to network Round-Trip Time (RTT) overhead. Pipelining groups multiple commands into a single TCP packet. Under a pipeline depth of 16, the system handles 81.3k RPS for write (SET) operations and 87.4k RPS for read (GET) operations. This confirms that the bottleneck resides in I/O syscalls rather than concurrency locks or data-structure operations. The kqueue Reactor efficiently aggregates input buffers and dispatches batch frames in a single reactor loop, minimizing context-switching costs.

3.2 Lock Contention & Sharding

To avoid thread-contention bottlenecks on a single global database lock, the store is partitioned into 16 independent shards, determined via DJB2 hashing. Each shard utilizes a `std::shared_mutex`. Multiple read (GET) commands on the same shard acquire shared locks (Readers-Writer lock pattern), allowing concurrent reads without locking out other threads. Write (SET/DEL) commands obtain exclusive locks. This sharding strategy effectively scales throughput with the number of CPU cores.

4. Key Upgrades: C vs. C++ Project Comparison

Dimension	Legacy C Project (ev_kv_store)	New C++ Project (redis_cpp)
Memory Management	Manual malloc/free, high risk of leaks & double-free.	Strict RAI, move-only wrappers, smart pointers.
Event-Loop APIs	epoll/kqueue with platform-dependent macros.	Clean abstraction inside EpollReactor class.
TTL / Expiry	Periodic linear O(N) scan of all database keys.	Min-heap O(log N) scan; wakes exactly on deadline.
Persistence	None. Data is lost completely on shutdown.	Append-Only File (AOF) with startup replay/recovery.
Command Architecture	Hardcoded switch/case block with line tokenization.	Clean polymorphic Command Interface (Command Pattern).
Concurrency Primitives	Pthread mutexes (exclusive locking only).	std::shared_mutex supporting SWMR (Single Writer Multi Reader).
Types & Generics	Void pointers, generic casting, poor type safety.	Modern STL containers, templates, and std::optional.

5. Conclusion & Recommendations

The redis_cpp project demonstrates successful design patterns in modern C++ systems programming.

1. Recommendations for CV / Portfolio:
 - Highlight the transition from C to C++, emphasizing the design shift from procedural C structures to Object-Oriented patterns like the Command Pattern and RAI.
 - Emphasize the benchmark numbers: Over 87k RPS for GET operations natively on macOS and a min-heap implementation for TTL management that reduces complexity from O(N) to O(log N).
 - Mention the AOF recovery subsystem, demonstrating full lifecycle persistence design.
2. Future Roadmap Extensions:
 - Implement client pipelining validation inside tests.
 - Introduce memory limits and LRU/LFU cache eviction policies.
 - Extend the command set to support Redis Data Structures like Lists, Sets, and Hashes.